CamComposite

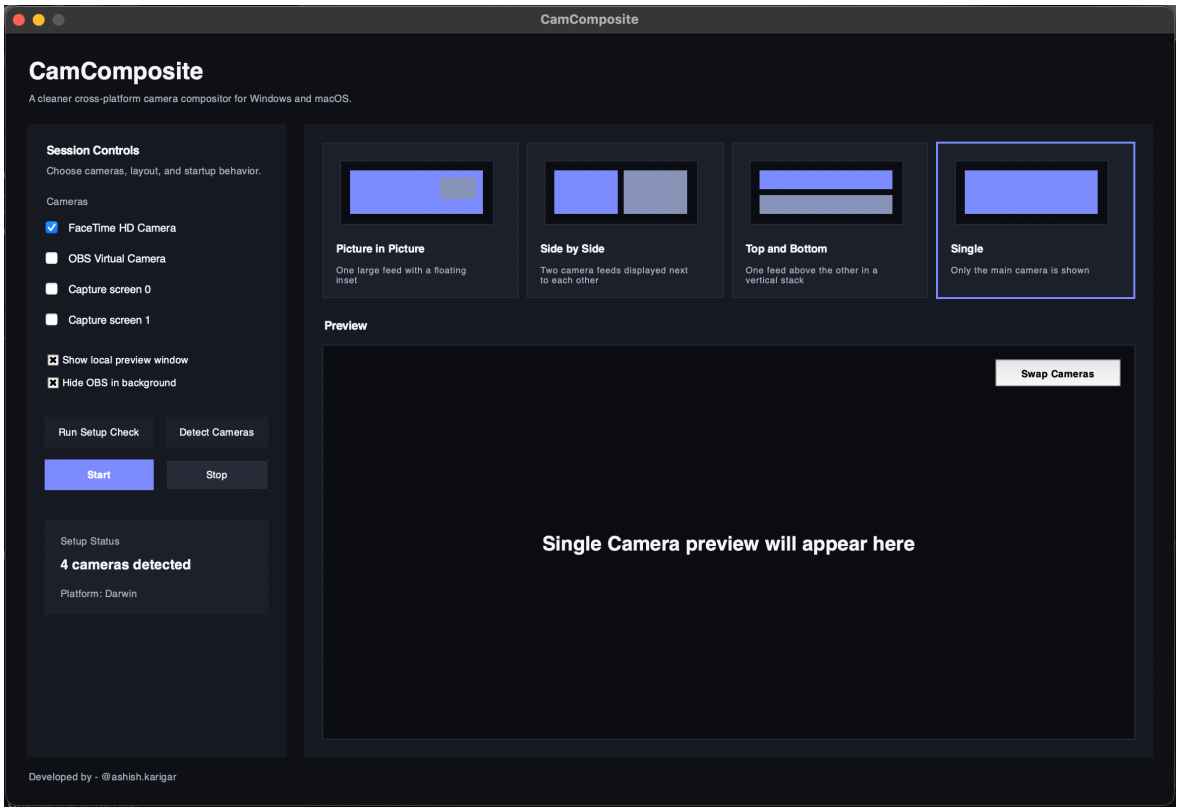
Abstract

CamComposite is a desktop application for combining one or two camera feeds into a single live composite output for video conferencing and related workflows. It lets the user detect available cameras, select up to two inputs, preview the combined layout, and publish the result as a virtual camera feed that other apps can use.

The app is designed for both macOS and Windows, with each platform using its own virtual-camera and dependency setup path:

- **macOS** uses **OBS + DistroAV + NDI runtime** to route the composited feed.
- **Windows** uses **UnityCapture** with `pyvirtualcam` to publish the composited feed.

CamComposite includes a startup splash screen that checks platform-specific requirements on first launch and helps install missing components before opening the main UI.



How CamComposite Works

macOS flow

1. User launches CamComposite.
2. Startup splash checks whether required macOS components are available.
3. If OBS is missing, CamComposite installs it.
4. If the DistroAV OBS plugin is missing, CamComposite installs it.
5. If the NDI runtime is missing, CamComposite installs it.
6. If the required OBS scene configuration is missing, CamComposite copies it.
7. Main UI opens.
8. User selects camera inputs and layout.
9. CamComposite composites the feeds.
10. OBS/NDI path makes that composite available to conferencing apps.

Windows flow

1. User launches CamComposite.
2. Startup splash checks whether UnityCapture is installed and registered.
3. If UnityCapture is missing, CamComposite downloads or clones the required backend into `packaging/win/UnityCapture` and launches the installer with administrator elevation.

4. Once UnityCapture is available, the main UI opens.
5. User selects camera inputs and layout.
6. CamComposite composites the feeds.
7. `unity_frame_sender.py` publishes the composited frame using `pyvirtualcam` with the `unitycapture` backend.
8. Conferencing apps can use the virtual camera feed.

User Guide

Getting the installer from GitHub Releases

Download the installer from the **Releases** section of the GitHub repository.

macOS

Look for a file like:

`CamComposite-mac-v1.0.0.pkg`

Windows

Look for a file like:

`CamComposite-win-v1.0.0.exe`

Always download the installer that matches your operating system.

Installing and using CamComposite on macOS

What to expect

When you open CamComposite on macOS, the startup splash may check for and install:

- OBS
- DistroAV
- NDI Runtime
- CamComposite OBS scene configuration

Permissions you may see

Depending on your Mac and system settings, you may be prompted for:

- administrator password
- permission to install packages
- camera permission

- microphone permission, if your workflow uses it
- permissions related to OBS or virtual-camera routing

Installation steps

1. Download the latest macOS installer from GitHub Releases.
2. Open the `.pkg` file.
3. Follow the macOS installer prompts.
4. Launch CamComposite from Applications.
5. On first launch, allow the startup checks to complete.
6. Approve any required package installation or admin prompts.
7. If macOS asks for camera access, allow it.
8. Once setup completes, the main UI will open.

First launch behavior on macOS

On first launch, CamComposite may install missing components. This is expected. The splash screen will show status messages and progress logs while setup runs.

After installation

Once the main UI opens:

1. Select available cameras.
2. Choose a layout.
3. Start the virtual-camera workflow.
4. In your conferencing software, choose the appropriate routed virtual-camera source.

Installing and using CamComposite on Windows

What to expect

When you open CamComposite on Windows, the startup splash checks for **UnityCapture**.

If UnityCapture is not installed, CamComposite will:

1. place the UnityCapture backend under `packaging/win/UnityCapture`
2. launch the UnityCapture installer
3. request administrator approval through UAC if needed
4. continue once the backend is registered successfully

Permissions you may see

Depending on your machine, you may be prompted for:

- Windows UAC administrator approval
- camera permission

- antivirus / SmartScreen confirmation in some environments

Installation steps

1. Download the latest Windows installer from GitHub Releases.
2. Open the `.exe` installer.
3. Follow the setup wizard.
4. Launch CamComposite from the Start Menu or desktop shortcut.
5. On first launch, allow the UnityCapture check to complete.
6. If a UAC prompt appears, approve it.
7. If Windows asks for camera access, allow it.
8. Once setup completes, the main UI will open.

First launch behavior on Windows

The splash screen may report that UnityCapture is missing and begin installation. This is expected on systems where the backend has not yet been registered.

After installation

Once the main UI opens:

1. Select available cameras.
2. Choose a layout.
3. Start the composite feed.
4. In Zoom, Teams, Meet, or another conferencing app, select the CamComposite / UnityCapture virtual camera.

Developer Guide

Cloning the repository

Clone the project with Git:

```
git clone <your-repository-url>
cd CamComposite
```

The repository uses Git LFS for release artifacts or large files, make sure Git LFS is installed first and then run:

```
git lfs install
git lfs pull
```

Project structure

```
CamComposite/
├── .gitattributes
├── .gitignore
│
├── assets/
│   ├── CamComposite-OBS.json
│   ├── logo.webp
│   ├── bin/
│   │   └── macos/
│   │       └── ffmpeg
│   └── icons/
│       ├── CamComposite.ico
│       ├── CamComposite.icns
│       ├── CamComposite.png
│       └── CamComposite.iconset/
│
├── build-scripts/
│   ├── build_mac.sh
│   ├── build_win.ps1
│   ├── camcomposite_mac.spec
│   └── camcomposite_win.spec
│
├── packaging/
│   ├── mac/
│   │   ├── component.plist
│   │   ├── resources/
│   │   └── scripts/
│   └── win/
│       ├── CamComposite.iss
│       └── UnityCapture/
│
├── project-setup/
│   ├── requirements-mac.txt
│   └── requirements-win.txt
│
├── release/
│   └── <generated installers>
│
└── src/
    ├── app.py
    ├── constants.py
    ├── main.py
    ├── styles.py
    ├── helpers/
    ├── prototypes/
    ├── services/
    ├── ui/
    └── utils/
```

Setting up the project

1. Clone the repository

```
git clone <your-repository-url>
cd CamComposite
```

3. Install Python dependencies

macOS

```
pip install -r project-setup/requirements-mac.txt
```

Windows

```
pip install -r project-setup/requirements-win.txt
```

OS-specific runtime requirements

macOS runtime requirements

CamComposite on macOS depends on these components at runtime:

- OBS
- DistroAV plugin
- NDI Runtime

The app startup flow checks and installs these as needed.

Windows runtime requirements

CamComposite on Windows depends on:

- UnityCapture

The app startup flow checks and installs it as needed.

Running the project in development

macOS

```
python src/main.py
```

Windows

```
python src/main.py
```

On first run, the splash screen will perform platform-specific dependency checks.

Build scripts

macOS build script

Use:

```
./build-scripts/build_mac.sh
```

What happens:

1. script prompts for a version number
2. old build artifacts are removed
3. PyInstaller builds the `.app`
4. macOS packaging tools build the installer package
5. final installer is moved into `release/`
6. temporary packaging files are cleaned
7. final package is added to git

Expected output file:

`release/CamComposite-mac-v<version>.pkg`

Windows build script

Use:

```
powershell -ExecutionPolicy Bypass -File .\build-scripts\build_win.ps1
```

What happens:

1. script prompts for a version number
2. old build artifacts are removed
3. PyInstaller builds the Windows app folder
4. Inno Setup builds the Windows installer
5. final installer is moved into `release/`
6. temporary build files are cleaned
7. final installer is added to git

Expected output file:

`release/CamComposite-win-v<version>.exe`

Requirements to run the build scripts

macOS build requirements

To build the macOS installer successfully, you need:

- Python environment with project dependencies installed
- PyInstaller
- macOS packaging tools such as `pkgbuild` and `productbuild`

These are typically available on macOS developer environments.

Windows build requirements

To build the Windows installer successfully, you need:

- Python environment with project dependencies installed
- PyInstaller
- Inno Setup 6

Install Inno Setup with:

```
winget install --id JRSoftware.InnoSetup -e -s winget -i
```

Which file goes where

`release/`

Stores final distributable installers that are ready to upload to GitHub Releases.

Examples:

```
release/CamComposite-mac-v1.0.0.pkg
release/CamComposite-win-v1.0.0.exe
```

`build-scripts/`

Stores project build scripts and PyInstaller spec files.

`packaging/mac/`

Stores macOS packaging resources and installer support files.

`packaging/win/`

Stores Windows packaging files such as the Inno Setup script and UnityCapture backend folder.

`assets/`

Stores icons, logos, config files, and packaged binary assets.

`src/`

Stores the application source code.

Notes for developers

- When running either build script, you will be asked to enter a version number.
- That version number is used to name the final release artifact.
- Build folders like `build/` and `dist/` are temporary and are cleaned after a successful packaging run.
- The `release/` folder is the place to look for the final installer artifact after a build completes.

Typical developer workflow

1. Clone the repository.
2. Create and activate a virtual environment.
3. Install the OS-specific requirements file.
4. Run the app locally with `python src/main.py`.
5. Make code changes.
6. Test again locally.
7. Run the build script for your platform.
8. Enter the version number when prompted.
9. Collect the final installer from `release/`.
10. Upload the artifact to GitHub Releases if needed.